

Einsendeaufgabe MET-E1: Metriken

1. Tool 1: Code-Metriken mit PMD

Einrichtung und Hürden

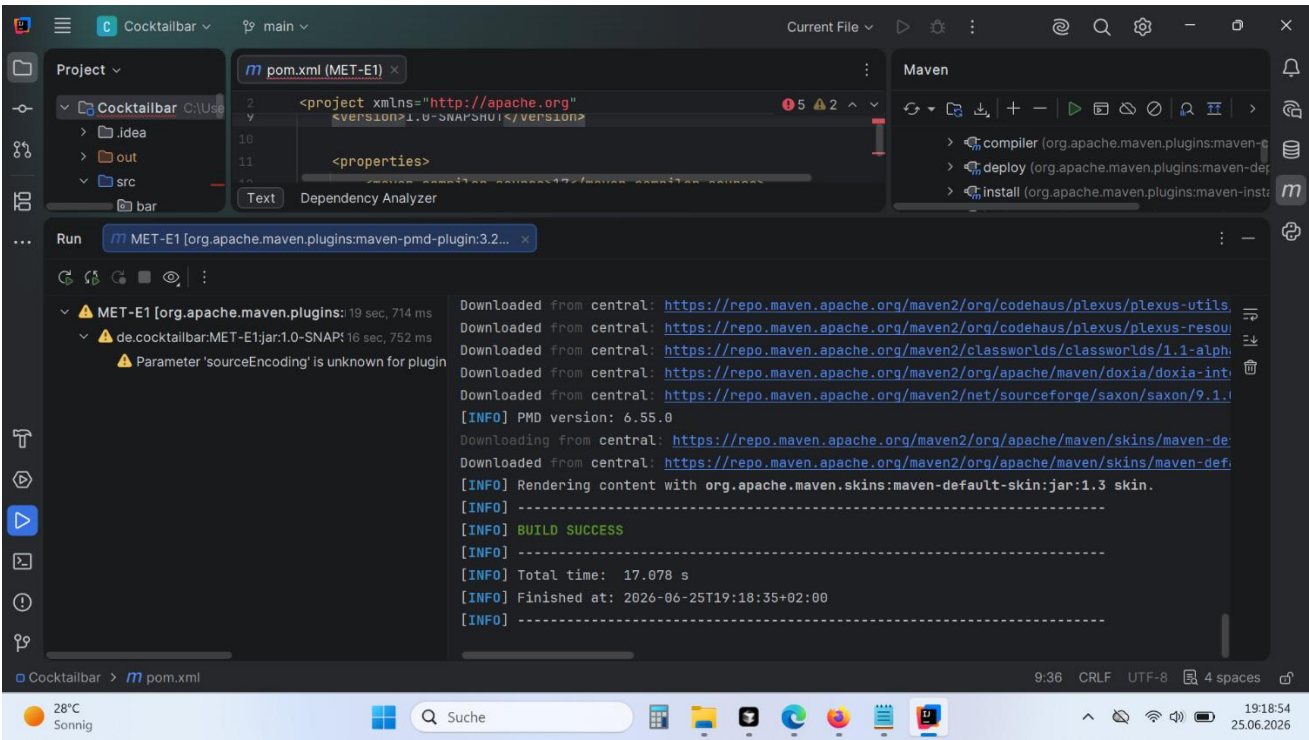
PMD wurde als maven-pmd-plugin in der Version 3.21.2 in die pom.xml integriert.

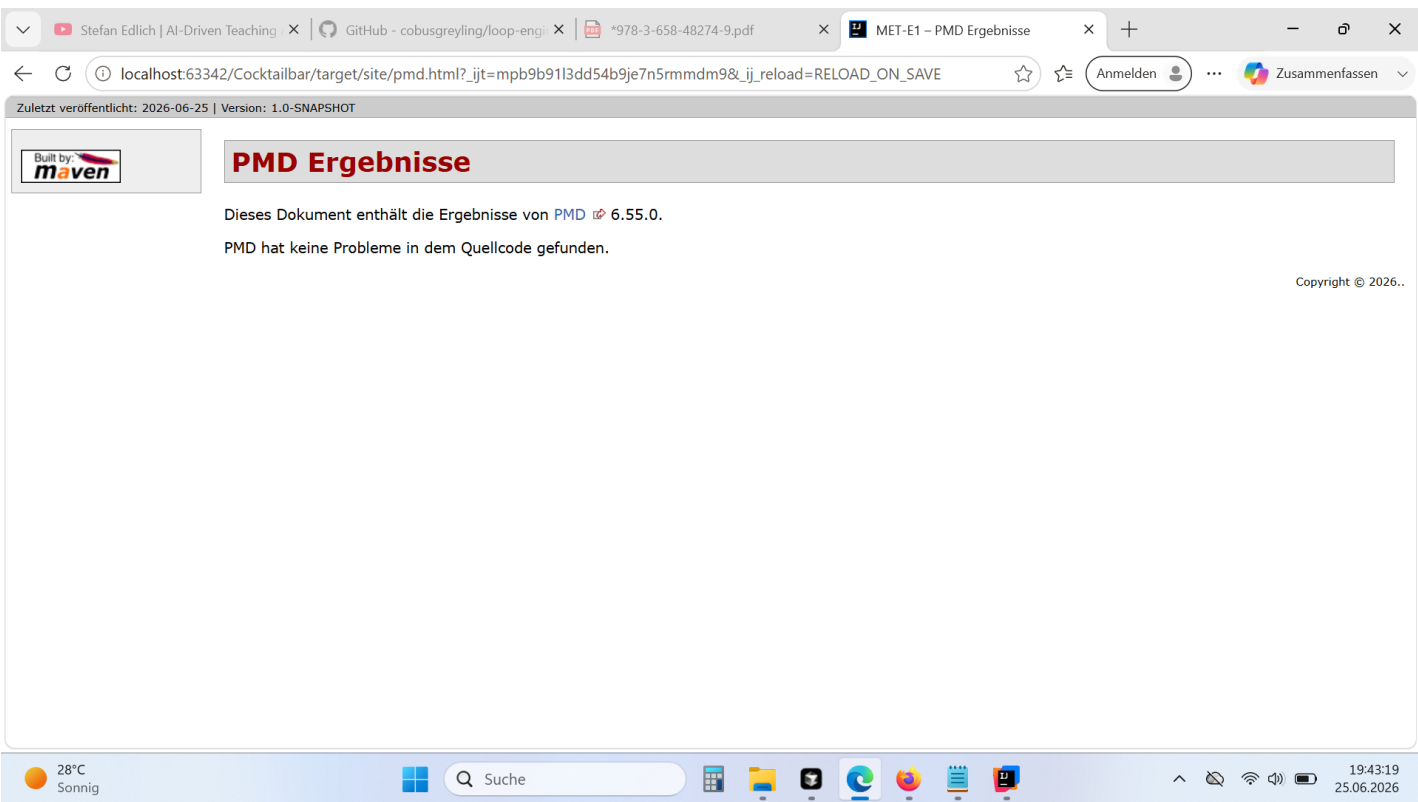
- **Problem:**Der Aufruf über das Terminal mittels mvn pmd:pmd schlug unter Windows fehl, da Maven nicht im Systempfad registriert war.
- **Lösung:** Das Problem gelöst, indem die grafische Benutzeroberfläche von IntelliJ genutzt wurde. Über das Maven-Tool-Fenster (Plugins -> pmd -> pmd:pmd) konnte die Analyse erfolgreich gestartet werden.

Ergebnisse und Interpretation

Die Analyse lief fehlerfrei durch, PMD generierte einen HTML-Report unter target/site/pmd.html.

- **Ergebnis:** „PMD hat keine Probleme in dem Quellcode gefunden.“
- **Interpretation:** Der Quellcode der Cocktailbar hält sich strikt an die gängigen Java-Codierungsrichtlinien (Code Smells, ungenutzte Variablen oder suboptimale Strukturen liegen nicht vor).





3. Tool 2: Architektur-Metriken mit JDepend

Kritische Hürden und Fehlerbehebung

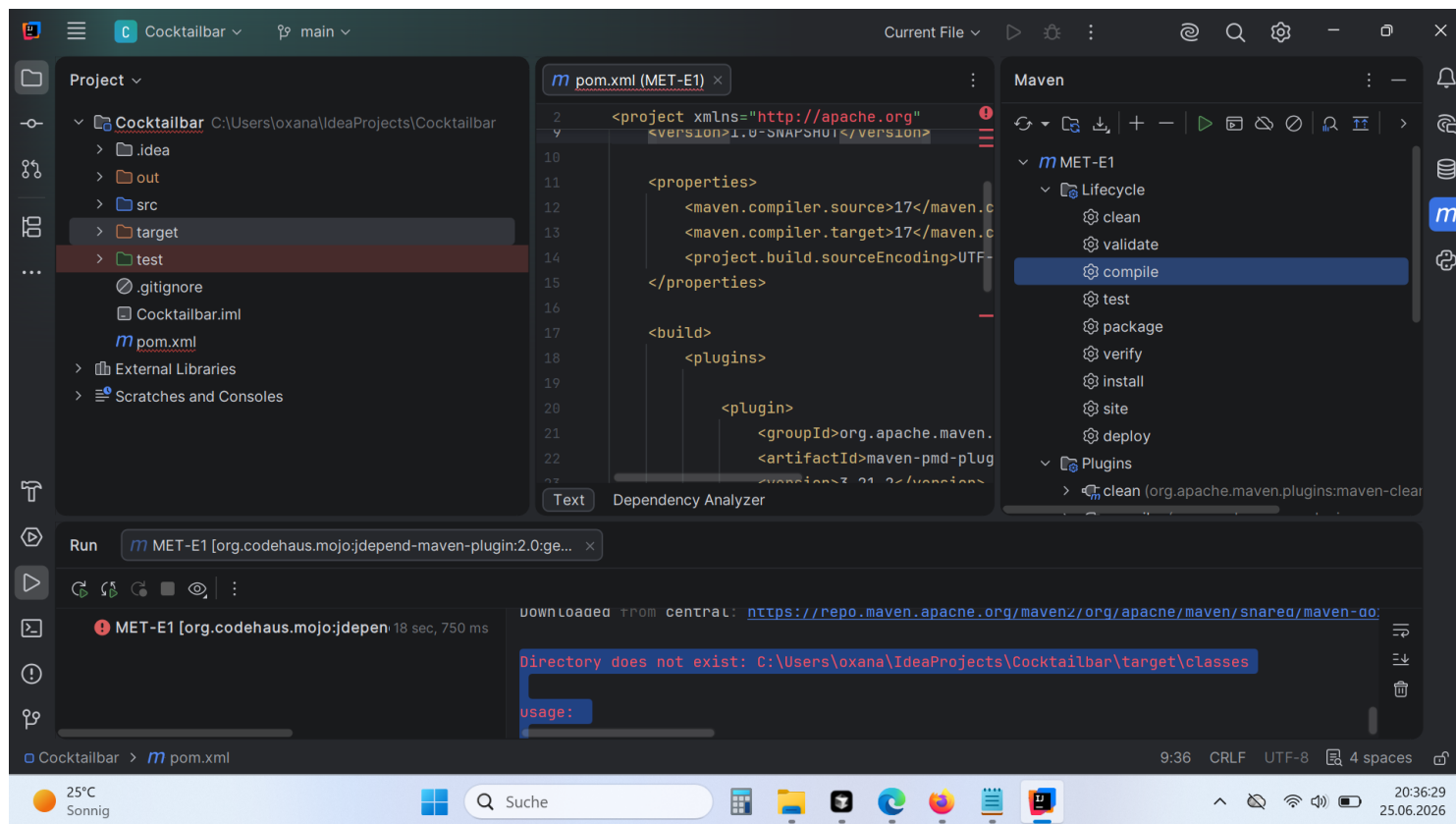
Die Einrichtung von JDepend über das jdepend-maven-plugin lieferte die größte Herausforderung

Fehler 1: Fehlender Klassen-Ordner

Problem: Der Aufruf von jdepend:generate brach mit der Fehlermeldung ab: Directory does not exist: ...\\target\\classes.

Ursache: JDepend analysiert nicht den rohen Quellcode, sondern den kompilierten Java-Bytecode (class-Dateien). Da das Projekt zuvor nie gebaut wurde, existierte der Ordner nicht.

Lösung: Ausführung des Maven-Lifecycles compile vor dem JDepend-Lauf.



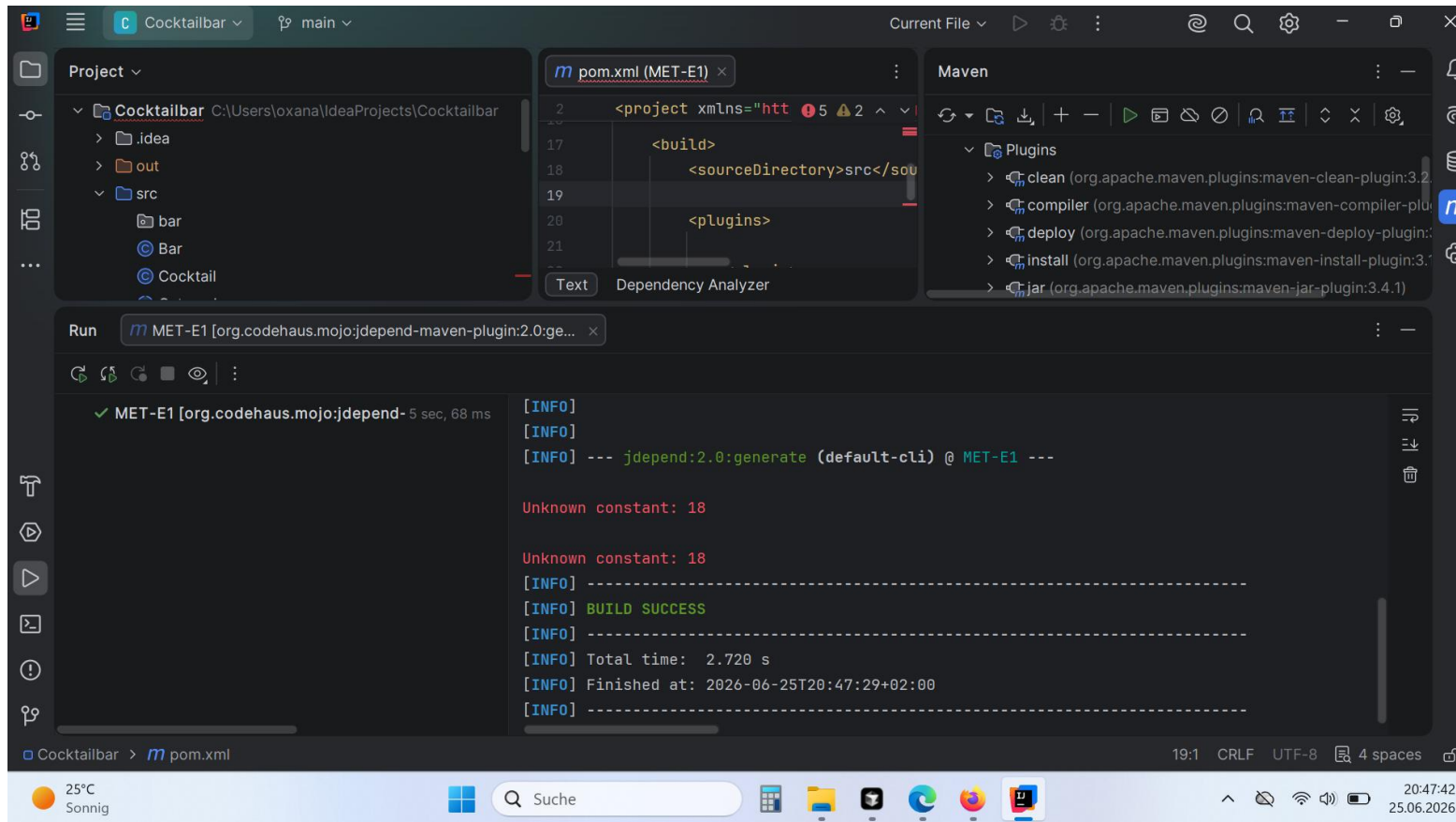
2. Fehler 2: Maven Standard-Layout-Konflikt

Problem: Auch nach dem Kompilieren blieb der Fehler bestehen. Maven meldete „No sources to compile“.

Ursache: Das Projekt nutzt eine flache Ordnerstruktur, Maven sucht Quellcode standardmäßig aber zwingend unter src/main/java.

Lösung: In der pom.xml wurde das Quellcodeverzeichnis über das Tag `<sourceDirectory>src</sourceDirectory>` überschrieben. Nach einem Maven-Reload lief das Tool fehlerfrei durch.

Fehlerauszug: [INFO] >>> jdepend:2.0:generate (default-cli) > compile @ MET-E1 >>> [INFO] [INFO] --- resources:3.3.1:resources (default-resources) @ MET-E1 --- [INFO] skip non existing resourceDirectory C:\Users\oxana\IdeaProjects\Cocktailbar\src\main\resources [INFO] [INFO] --- compiler:3.13.0:compile (default-compile) @ MET-E1 --- [INFO] No sources to compile [INFO] [INFO] <<< jdepend:2.0:generate (default-cli) < compile @ MET-E1 < [INFO] [INFO] [INFO] --- jdepend:2.0:generate (default-cli) @ MET-E1 --- Directory does not exist: C:\Users\oxana\IdeaProjects\Cocktailbar\target\classes usage: JDepend [-components <components>] [-file <output file>] <directory> [directory2 [directory3] ...]



Ergebnisse und Interpretation

Der Report wurde als jdepend-report.xml generiert. Relevante Kennzahlen für das Paket bar und src:

- **Afferent Couplings Ca:** die Stabilität; wie viele andere Pakete hängen von diesem ab.
- **Efferent Couplings Ce:** die Kopplung; von wie vielen anderen Paketen hängt dieses ab.
- **Abstractness A & Instability I:** Da der Code primär aus konkreten Klassen wie Bar, Cocktail besteht und keine Interfaces nutzt, läuft die Abstraktion gegen 0, während die Instabilität (Kopplungen) mathematisch berechnet wird.

The screenshot shows an IDE with a project named 'Cocktailbar' located at 'C:\Users\oxana\IdeaProjects\Cocktailbar'. The project structure includes folders like '.idea', 'out', 'src', 'target', 'classes', 'generated-sources', 'maven-status', 'pmd', and 'site'. The 'target' folder is expanded, showing 'classes', 'generated-sources', 'maven-status', 'pmd', and 'site'. The 'jdepend-report.xml' file is selected in the 'target' folder. The IDE also shows 'pom.xml' and 'test' files. The 'Run' tab at the bottom shows 'MET-E1 [org.codehaus.mojo:jdepend-maven-plugin:2.0:ge...]'.

The 'jdepend-report.xml' file content is as follows:

```
<?xml version="1.0"?>
<JDepend>
  <Packages>
    <Package name="bar">
      <Stats>
        <TotalClasses>4</TotalClasses>
        <ConcreteClasses>2</ConcreteClasses>
        <AbstractClasses>2</AbstractClasses>
        <Ca>0</Ca>
        <Ce>2</Ce>
        <A>0.5</A>
        <I>1</I>
        <D>0.5</D>
        <V>1</V>
      </Stats>
      <AbstractClasses>
        <Class sourceFile="Getraenk.java">
          bar.Getraenk
        </Class>
        <Class sourceFile="RezeptFilter.java">
          bar.RezeptFilter
        </Class>
      </AbstractClasses>
    </Package>
  </Packages>
</JDepend>
```

4. Metrik-Berechnung McCabe-Komplexität

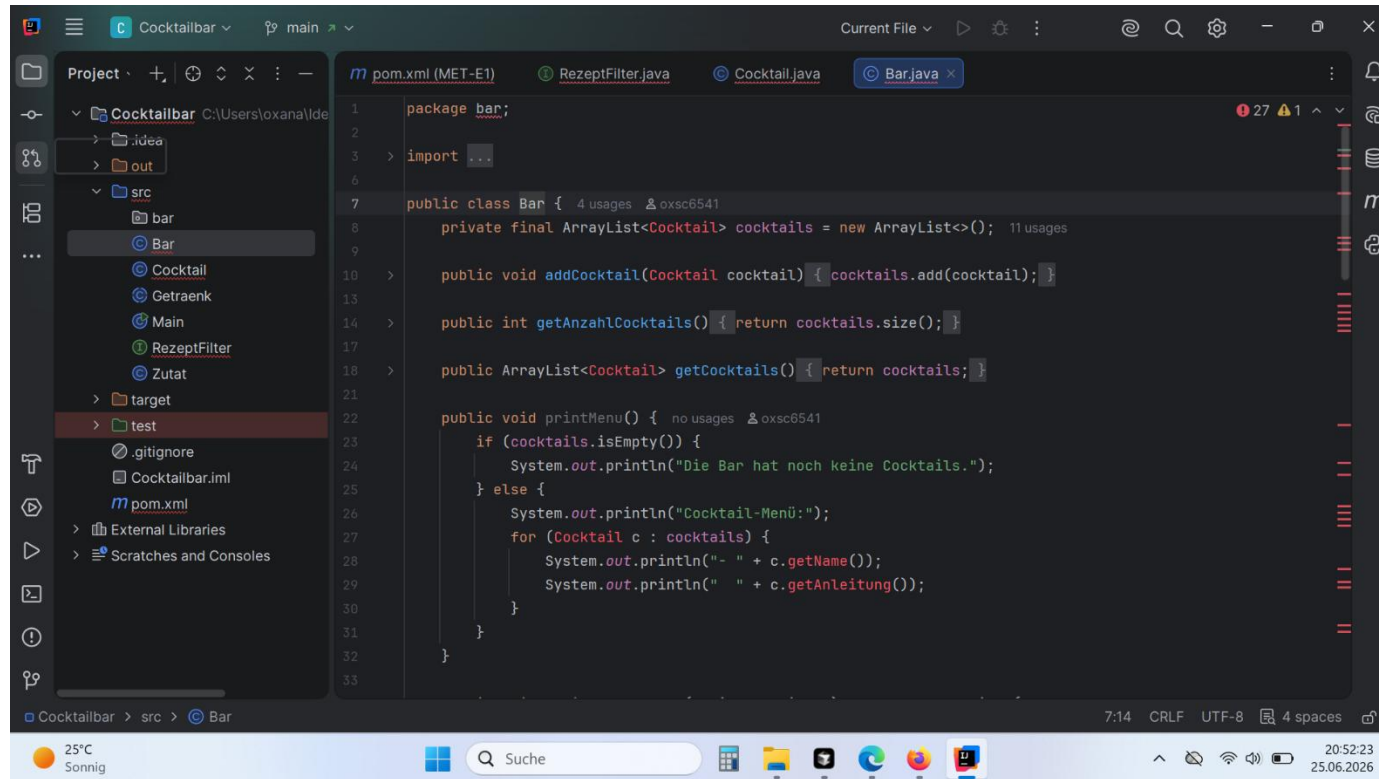
Code-Auszug:

```
public void printMenu() {
    if (cocktails.isEmpty()) {
        System.out.println("Die Bar hat noch keine Cocktails.");
    } else {
        System.out.println("Cocktail-Menü:");
    }
}
```

```

    for (Cocktail c : cocktails) {
        System.out.println("- " + c.getName());
        System.out.println(" " + c.getAnleitung());
    }
}
}
}

```



Für die Methode printMenu() in der Klasse Bar.java wurde die zyklomatische Komplexität nachgerechnet.

- **Formel:** $V(G) = P + 1$ (Entscheidungspunkte + 1)

- **Berechnung:**

Startwert : 1

Entscheidungspunkt if (cocktails.isEmpty()) (Zeile 23): +1

Entscheidungspunkt for (Cocktail c : cocktails) (Zeile 27): +1

- **Ergebnis:** Gesamte McCabe-Komplexität = 3

Interpretation: Der Wert 3 sagt, dass es drei linear unabhängige Pfade durch die Methode gibt (leere Bar, Bar mit Cocktails, Durchlauf der Schleifenelemente). Für eine vollständige Testabdeckung wären somit mind. 3 Testfälle notwendig.